3.3 Algorithm Justification

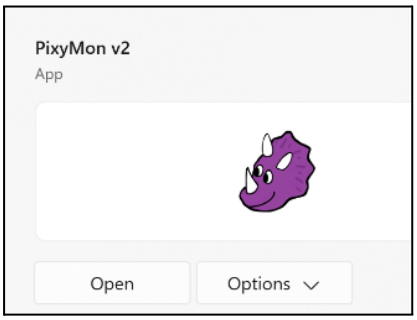
3.3.1 CV – Computer Vision (Pixy Camera Colour Detection)

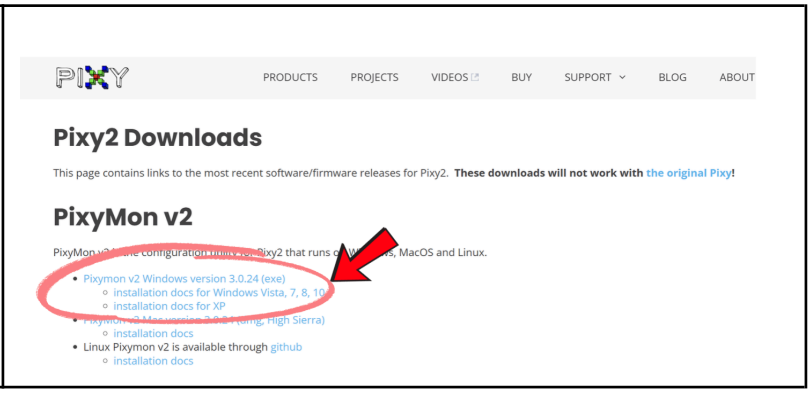
```
Function getSignature(in number port, in number
sig, out number x, out number y)
  address = 80 + sig
  values = Sensor.ReadI2CRegisters(port, 1,
address, 3)
  x = values[1]
  y = values[2]
EndFunction
```

```
While "True"
  getSignature(3,1,greenx,greeny)
  getSignature(3,2,redx,redy)
  getSignature(3,3,pinkx,pinky)
```

- **Multi-Colour Detection:** Enables simultaneous recognition of Green, Red and Pink pillars within a single camera frame.
- **Coordinate Extraction:** Provides individual X and Y coordinates for each detected pillar, allowing further position and distance calculations.
- **Color Signature Recognition:** Uses PixyMon to configure and store individual colour signatures for consistent object identification.
- **Detection Tuning:** Reduced Signature Range and performed repeated testing to minimise Red and Pink colour confusion.

Tuning & Testing Process

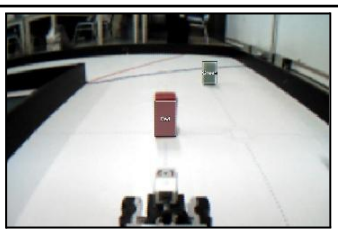
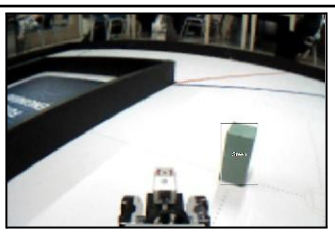
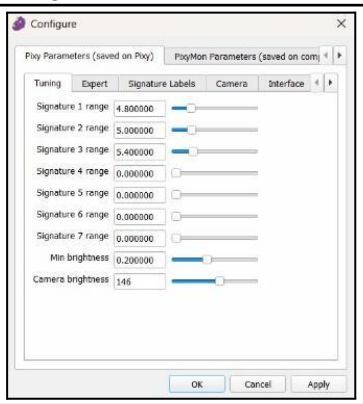
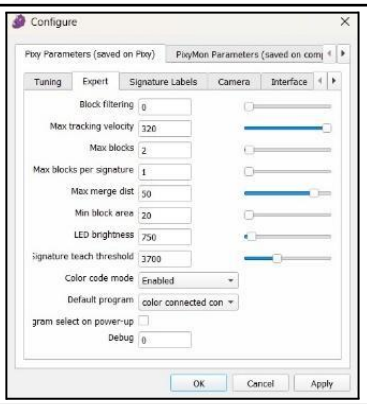




Refer to installation address:https://pixycam.com/downloads-pixy2/

The PixyMon application was used to configure the Pixy Camera's colour signatures and adjust its detection parameters.

PixyMon Camera Tuning

<table>
<tr>
<td> 
Set signature for both obstacles</td>
<td> 
Coordinate value : (158,55)</td>
</tr>
<tr>
<td> 
</td>
<td> 
</td>
</tr>
</table>

- Green, Red, and Pink Colour Signatures were configured individually using PixyMon.
- Signature Ranges were set to 4.8, 5.0, and 5.4, respectively.
- Configured the maximum tracking velocity to 320.
- Limited the maximum detected blocks to 2, with a maximum of 1 block per signature.
- Set the minimum block area to 20 to define the detection area.
- The detected Green Pillar coordinate was verified at (158, 55).

3.3.2 IMU – Gyroscope-Based Heading Detection and Correction

```
gyro = Sensor.ReadRawValue(2,0)
relativeHeading = gyro - @target
```

```
turn = (@relativeHeading * 1.3)
```

- **Heading Measurement:** Reads Gyroscope data using Sensor.ReadRawValue() to obtain the robot's angular position.
- **Heading Monitoring:** Continuously compares the current Gyroscope reading with the target heading.
- **Heading Correction:** Calculates heading error to generate steering adjustments.
- **Turning Compensation:** Uses 91° and 90.5° target increments to compensate for actual turning deviations.
- **Sensitivity Tuning:** Applies a correction coefficient of 1.3 to balance responsiveness and driving stability.

Heading Angle Tuning Table

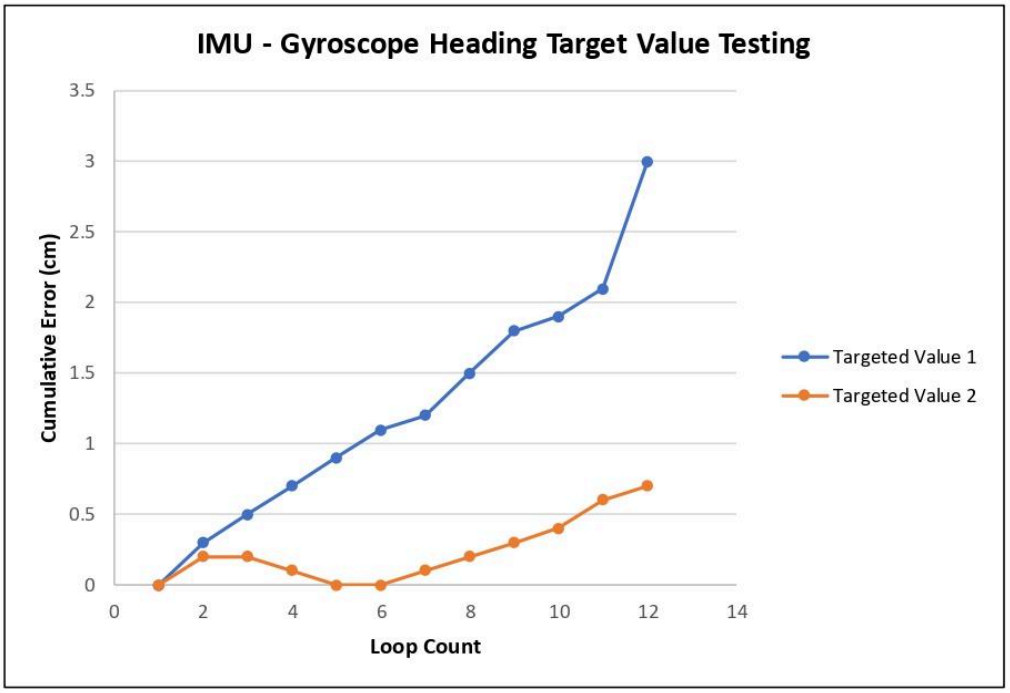
Parameter	Initial Configuration	Final Configuration
Gyroscope Target Angle	90°	91° (First Six Laps)
Subsequent Heading Increment	-	90.5° (Subsequent Laps)

Notes:

- The initial 90° target did not consistently produce an accurate 90° physical turn.
- The target angle was adjusted to 91° for the first six laps and 90.5° for subsequent laps to improve turning accuracy

IMU - Gyroscope Heading Target Value Testing

Notes: Measurement error was determined by measuring the distance (cm) between Point A, the robot's position after a 90° turn, and Point B, its final position after a 500° driving motor rotation. The distance between A and B represents the cumulative positional error caused by inaccurate 90° turning.



Notes:

- **Targeted Value 1 (90° for all 12 loops.)**
- **Targeted Value 2 (91° for the first 6 loops, followed by 90.5° for loops 7–12.)**

Engineering Decision

- Targeted Value 2 (91° for the first 6 loops, followed by 90.5° for loops 7–12) was

selected as the final IMU heading configuration.

- Targeted Value 2 achieved a lower average cumulative error of 0.85 cm, compared to 5.8 cm for Targeted Value 1.
- This represents an approximately 85.3% reduction in average cumulative error, demonstrating improved turning accuracy.
- Targeted Value 2 also maintained lower cumulative errors throughout the testing process, indicating better consistency and stability.
- Therefore, **Targeted Value 2 was chosen** as the most stable and effective configuration to minimise positional errors and improve the robot's overall navigation performance.

3.3.3 PID – PID Steering Control Algorithm

```
Function PID(in number turn, in number kp, in number ki, in number kd)
  error = (49 + turn) - Motor.GetCount("A")

  p = error * kp

  @intergal = @intergal + error
  If @intergal>5 Then
    @intergal = 5
  ElseIf @intergal<-5 Then
    @intergal=-5
  ElseIf error=0 Then
    @intergal = 0
  EndIf
  i = @intergal*ki

  d = (error-@lastError)*kd

  result = p+i+d
  Motor.StartPower("A",result)
  If result < 40 Then
    result = 38
  EndIf
  @lastError = error
EndFunction
```

Closed-Loop Control: Uses Steering Motor Encoder Feedback to compare the target steering position with the actual motor position.

- **Proportional Correction (P):** Adjusts steering output according to the current position error.
- **Integral Correction (I):** Accumulates previous errors to address persistent steering deviations.
- **Derivative Correction (D):** Responds to changes in steering error to reduce oscillation and improve steering stability.
- **Integral Windup Prevention:** Limits the accumulated integral value between -5 and +5 to prevent excessive integral accumulation.
- **Minimum Motor Power:** Designed to provide sufficient motor power to overcome mechanical resistance and prevent ineffective steering movement.

Tuning and Testing Process

Parameter	Initial Value	Final Value	Testing Observation
Kp	3	5	Stronger steering correction response
Ki	-	0.1	Improved steering position by adjusting minor

			error for accuracy and driving stability
Kd	5	4	Reduced excessive adjustment frequency and improved driving smoothness

- **Final Configuration:** The PID parameters were set to Kp = 5, Ki = 0.1, and Kd = 4.

3.3.4 Mathematical Obstacle Detection Algorithm

```
'------------------Hypothaneuse area calculation------------------'
greenDis = Math.SquareRoot(Math.Power(@greenx-130,2)+Math.Power(255-@greeny,2))
redDis = Math.SquareRoot(Math.Power(@redx-130,2)+Math.Power(255-@redy,2))
pinkDis = Math.SquareRoot(Math.Power(@pinkx-130,2)+Math.Power(255-@pinky,2))
```

Two-Dimensional Distance Calculation: Uses both X and Y coordinates to determine the relative position of detected pillars.

- **Euclidean Distance:** Applies the Pythagorean Theorem to calculate the distance between each obstacle and the camera's bottom-centre reference point.
- **Consistent Reference Point:** Uses (130,255) as a common reference for distance comparison.
- **Nearest Obstacle Identification:** Compares Green and Red Signature distances to identify the nearest obstacle.
- **Distance Threshold:** Activates obstacle handling only when a detected Pillar enters its predefined distance threshold, preventing unnecessary steering responses to distant obstacles.

3.3.5 Mathematical Obstacle Tracking and Avoidance

```
    If  (greenDis<@greenH Or redDis<@redH Or pinkDis<130) And @trackPillar=1 And logic=1 Then
'show LED color 'value bigger, farer can see
      If greenDis<redDis And greenDis<pinkDis Then
        @lastPillar = 1
        EV3.SetLEDColor("GREEN","NORMAL")
        If @greeny>@gAvoid Then '<-----------------------------------Green pillar y (value big,
nearer) '
          turn = (@greenAvoid-@greenx)*@greeny/100*0.5 'avoid pillar
          'Speaker.Tone(30,2000,5)
        Else
          turn = (@greenFollow-@greenx)*@greeny/100*0.7 'keep car in the middle
          Speaker.Tone(20,100,50)
        EndIf
      ElseIf redDis<greenDis And redDis<pinkDis Then
        @lastPillar = 2
        EV3.SetLEDColor("RED","NORMAL")
        If @redy>@rAvoid Then'<-----------------------------------Red pillar y (value big,
nearer) '
          turn = (@redAvoid-@redx)*@redy/100*0.5 'avoid pillar
          'Speaker.Tone(30,1000,5)
        Else
          turn = (@redFollow-@redx)*@redy/100*0.7 'keep car in the middle
          Speaker.Tone(20,1000,50)
        EndIf
```

- **Position-Based Detection:** Uses the Pillar's X and Y Coordinates to determine its position and decide when to transition from Following to Avoidance.
- **Mathematical Steering Calculation:** Calculates horizontal deviation from the target X Position and incorporates the Y Coordinate to generate steering adjustments.
- **Parameter Optimization:** Uses different steering coefficients (0.7 for Following and 0.5

for Avoidance), calibrated through repeated testing.

- **Directional Calibration:** Uses different target X Positions for CW and CCW navigation to accommodate opposite driving directions.
- **Continuous Navigation:** Continuously detects Green and Red Pillars and uses Gyroscope feedback to monitor heading after obstacle avoidance.

Tracking Target Value Tuning

Notes: X-coordinate ranges from **0 (far left) to 255 (far right)**. The tested values are near the centre, slightly to the left or right, for tracking green and red pillars.

Variable	Tested Targeted Value (X- Coordinate)	Successful Attempts	Accuracy
Green Pillar Tracking (greenFollow)	120	6/10	60%
	130	8/10	80%
	140	**10/10**	**100%**
	150	7/10	70%
Red Pillar Tracking (redFollow)	110	6/10	60%
	120	**9/10**	**90%**
	130	8/10	80%
	140	7/10	70%

The tracking target coordinates were tested using different values to evaluate the robot's ability to maintain its intended path around coloured pillars.

The results showed that **greenFollow = 140 achieved 100% accuracy,** while **redFollow = 120 achieved 90% accuracy.**

These values were selected because they provided the most consistent tracking performance during testing.

Avoidance Threshold Value Tuning

Notes: Y-coordinate indicates the pillar's vertical position in pixels. When the detected Y-coordinate reaches the predefined threshold, the robot initiates the avoidance maneuver.

Variable	Tested Targeted Value (Y-Coordinate)	Successful Attempts	Accuracy
Green Avoid Distance (gAvoid)	110	6/10	60%
	120	8/10	80%
	130	**10/10**	**100%**

	140	7/10	70%
Red Avoid Distance (rAvoid)	90	6/10	60%
	100	8/10	80%
	110	**10/10**	**100%**
	120	7/10	70%

- The avoidance trigger thresholds were tested using different Y-coordinate values to determine the appropriate distance for initiating obstacle avoidance.
- Based on the testing results, **gAvoid = 130 and rAvoid = 110** achieved the highest accuracy of **100% i**n their respective tests.
- These values were selected to ensure timely obstacle avoidance while maintaining smooth navigation.

Avoidance Target Value Tuning

Notes: X-coordinate indicates the pillar's horizontal position in pixels, ranging from 0 (left) to 255 (right). The tested value is subtracted from the current X-coordinate to determine the avoidance target position for green and red pillars.

Variable	Tested Targeted Value (X-Coordinate)	Successful Attempts	Accuracy
Green Pillar Avoidance (greenAvoid)	225	6/10	60%
	235	8/10	80%
	245	**10/10**	**100%**
	255	7/10	70%
Red Pillar Avoidance (redAvoid)	5	6/10	60%
	10	**9/10**	**90%**
	15	8/10	80%
	20	7/10	70%

- The avoidance target coordinates were tested using different X-coordinate values to determine the most effective steering positions for passing around coloured pillars.
- The results showed that **greenAvoid = 245 achieved 100% accuracy**, while **redAvoid = 10 achieved 90% accuracy**.
- These values were selected because they provided the best obstacle clearance performance during testing, reducing the risk of collision while maintaining the intended navigation path.

3.3.6 Wall Following Algorithm

```
If Math.Abs(@relativeHeading) < 15 Then '-----------if langgar wall car
--------'
  If @cw=1 Then
    followWall = (@leftWall - @wallDistance) * @followWallResponse
  Else
    followWall = (@rightWall - @wallDistance) * -@followWallResponse
  EndIf
Else
  followWall = 0
EndIf
```

- **Wall-Based Navigation:** Uses the surrounding wall as a reference to maintain a stable driving path.
- **Distance Control:** Maintains a predefined target distance from the wall to reduce lateral deviation.
- **Gyroscope Integration:** Complements Gyroscope heading correction by adjusting the robot's lateral position relative to the wall.
- **Practical Application:** Supports three-lap navigation in the Open Challenge and guides the robot towards the Parking Lot in the Obstacle Challenge.
- **Stage-Based Parameter Tuning:** Uses different wallDistance and followWallResponse values for your different driving stages to achieve suitable navigation performance.

3.3.7 Wall Avoidance Algorithm

```
  If (@leftWall<150 Or @rightWall<150) And @trackWall=1 And
@Parking = 0  Then 'sometimes too near to wall, the value will be
2550
    If @leftWall<150 Then
      turn = -30'-------------------steering reaction--------'
      Speaker.Tone(50,800,600)
    ElseIf @rightWall<150 Then
      turn = 30
      Speaker.Tone(50,200,600)
    EndIf
  Else
```

Threshold Calibration: Sets the Wall Avoidance Threshold to 150 based on practical testing, identifying when the robot is too close to the wall.

- **Direct Steering Response:** Uses **fixed steering values of -30 and +30** to provide immediate and predictable steering adjustments.
- **Motor Range Consideration:** Selects steering values according to the Steering Motor's operating range to ensure suitable turning movements.

Tuning and Testing for Wall Avoidance Algorithm

Parameter	Tested Targeted Value	Observation	Evaluation
wallDistance	100mm	The robot travelled too close to the wall, leaving insufficient clearance.	Not Suitable
	125mm	The robot maintained a closer path but showed occasional lateral deviation.	Acceptable
	150mm	**The robot maintained a stable distance from the wall with sufficient clearance.**	**Selected**
	175mm	The robot travelled farther from the wall, reducing navigation consistency.	Not Suitable
	200mm	The robot moved too far away from the wall, affecting navigation accuracy.	Not Suitable
Left Steering Correction	-20	Steering response was too weak to correct lateral deviation effectively.	Not Suitable
	-25	Steering correction improved, but the response remained relatively slow.	Acceptable
	-30	**Provided a suitable steering response and maintained a stable path.**	**Selected**
	-35	Steering correction became more aggressive.	Not Suitable
	-40	Excessive steering correction caused noticeable deviation.	Not Suitable
Right Steering Correction	20	Steering response was too weak to correct lateral deviation effectively.	Not Suitable
	25	Steering correction improved, but the response remained relatively slow.	Acceptable
	30	**Provided a suitable steering response and maintained a stable path.**	**Selected**
	35	Steering correction became more aggressive.	Not Suitable
	40	Excessive steering correction caused noticeable deviation.	Not Suitable

- The selected parameters (wallDistance = 150 mm, left correction = -30, and right correction = +30) provided the most stable wall-following performance.
- It maintained sufficient wall clearance and reduced unnecessary steering adjustments, ensuring smoother and more consistent navigation.